

Senior Data Engineer

Take-Home Assessment

Product-Focused Data Engineering and Agentic Engineering

Estimated duration	3-4 hours for the core implementation, plus up to 30 minutes for the design note.
Submission	Git repository or zip archive containing source code, README, outputs, and design note.
Recommended stack	Databricks Community Edition, dbt Core/dbt Cloud-compatible project structure, Python/PySpark, and SQL. Alternatives are acceptable if documented.
Assessment goal	Show how you build reliable data pipelines, reason about data quality, and turn a technical solution into a trustworthy data product.

Please do not commit secrets, tokens, or credentials. API details should be read from environment variables, or a local config file excluded from version control.

What We Value

- ☐ Pragmatic engineering judgment over unnecessary complexity.
- ☐ Clear ownership of data quality, idempotency, failure handling, and operational readiness.
- ☐ Readable, reproducible work that another engineer could run and maintain.
- ☐ Product thinking: understanding who uses the data, how trust is established, and how the solution could become reusable.
- ☐ Responsible use of AI-assisted engineering tools, if you choose to use them.

1. Scenario

You are joining a product data platform team responsible for exposing trusted payment transaction data to analysts, product managers, and downstream data products. Today, ingestion is manual, data quality issues are discovered late, and downstream users do not fully trust the data.

Your task is to build a small but production-minded ingestion and transformation pipeline. The pipeline should fetch transactions from a REST API, validate and quarantine defective records, persist raw records, produce a daily account-level summary, and demonstrate incremental ingestion with watermark handling.

We are not looking for a large framework. We are looking for evidence that you can make pragmatic trade-offs, produce clean work, and explain how your solution would evolve into a reusable product/platform capability.

2. API and Configuration

The API base URL and credentials will be shared separately. Your code should not hardcode those values.

Environment variable	Purpose
ASSESSMENT_API_BASE_URL	https://fgbjekjqnbmtkmeewexb.supabase.co/rest/v1
ASSESSMENT_API_KEY	API Key used in the header apikey sb_publishable_W2MbiakvFFthMHtlrzSkQw_URTiUI6G
ASSESSMENT_AUTH_TOKEN	Bearer token used in the Authorization header. This may be the same value as the API key. Authorization: Bearer sb_publishable_W2MbiakvFFthMHtlrzSkQw_URTiUI6G

The API supports offset-based pagination and date filtering. Iterate until you receive an empty response or fewer records than the requested limit.

- ☐ GET /transactions
- ☐ GET /transactions?limit=M&offset=N
- ☐ GET /transactions?order=transaction_date.asc
- ☐ GET /transactions?account_id=eq.ACC-1001
- ☐ GET /transactions?transaction_date=gte.2024-02-01T00:00:00Z

3. Transaction Schema

Field	Type	Required	Rules
transaction_id	string	Yes	Application-level unique identifier. Format: TXN-NNNN.
account_id	string	Yes	Account identifier. Format: ACC-NNNN.

transaction_date	datetime	Yes	Strict ISO 8601 UTC timestamp with T separator and Z suffix.
amount	decimal	Yes	Must be strictly greater than zero.
currency	string	Yes	One of USD, EUR, GBP, CHF, JPY, AUD, CAD.
transaction_type	string	Yes	Lowercase only: debit or credit.
merchant_name	string	Yes	Non-empty and not whitespace-only.
merchant_category	string	Yes	One of e-commerce, travel, food_and_beverage, groceries, electronics, retail, entertainment, health, transportation, home_and_garden, payroll, transfer.
status	string	Yes	Lowercase only: completed, pending, failed, reversed.
country_code	string	Yes	Assigned ISO 3166-1 alpha-2 country code. Format alone is not sufficient.

4. Data Quality Conditions

The dataset intentionally contains defects. Your pipeline must detect, report, and isolate those records without crashing or silently dropping data.

- ☐ Invalid records: one or more fields violate the schema rules.
- ☐ Duplicate records: records represent the same real-world transaction but have different transaction_id values and identical values across all other fields.
- ☐ Duplicate detection must be based on the natural key across all fields except transaction_id.

5. Assignment Tasks

Task 1: Raw Ingestion with Data Quality Handling

Estimated time: 60-75 minutes

Build a pipeline that fetches all transactions and stores them in a raw/bronze layer while detecting invalid and duplicate records.

Requirements

- ☐ Fetch all records using pagination. Do not hardcode the record count.
- ☐ Read API settings from environment variables or a local config file excluded from version control.
- ☐ Implement retry logic with exponential backoff for HTTP 429, 5xx, and timeouts.

- ☐ Validate each record against the schema and enum rules.
- ☐ Route invalid records to a quarantine/dead-letter layer with `error_reason` and `ingestion_timestamp`.
- ☐ Detect duplicates by comparing all fields except `transaction_id`. Either flag or deduplicate them, and document your choice.
- ☐ Persist valid raw records as-is, plus ingestion metadata.

Deliverables

- ☐ Ingestion script or notebook.
- ☐ Raw and quarantine schema definitions.
- ☐ Quarantine sample output showing captured defects and reasons.
- ☐ Short notes on validation and duplicate handling.

Task 2: Daily Account Summary

Estimated time: 60-75 minutes

Build a transformation layer that produces a daily account-level summary for analysts and product stakeholders.

Requirements

- ☐ Only include records where `status = completed`.
- ☐ Exclude quarantined records from aggregations.
- ☐ Produce one row per `account_id` and `transaction_date`.
- ☐ The transformation must be idempotent: rerunning it should not change results unless input data changes.
- ☐ If using dbt, include `schema.yml` with at least three meaningful column tests. If not using dbt, include equivalent assertions or unit tests.

Deliverables

- ☐ Transformation SQL/model/script.
- ☐ Output sample for `daily_account_summary`.
- ☐ Tests, dbt `schema.yml`, or equivalent validation assertions.

Output fields for `daily_account_summary`

Field	Description
<code>account_id</code>	Account identifier.
<code>transaction_date</code>	UTC calendar date truncated from the transaction timestamp.

total_debit_amount	Sum of debit amounts for completed transactions.
total_credit_amount	Sum of credit amounts for completed transactions.
net_amount	total_credit_amount minus total_debit_amount.
transaction_count	Count of completed transactions.
distinct_merchants	Number of distinct merchants for the day.
top_category	Merchant category with highest total spend for the day.
currencies	Distinct currencies used that day, as an array or comma-separated string.
updated_at	UTC timestamp when the row was last computed.

Task 3: Incremental Ingestion and Watermark Logic

Estimated time: 60-75 minutes

Extend the ingestion pipeline so subsequent runs request and process only records newer than the last successful run.

Requirements

- ☐ Persist the high-water mark: maximum transaction_date successfully ingested.
- ☐ Use the API native date filter, for example transaction_date=gte.<watermark>.
- ☐ Demonstrate a second run where no duplicate rows are inserted and the pipeline completes safely.
- ☐ Explain first-run behavior, no-new-data behavior, and late-arriving data handling.
- ☐ State whether you use a lookback window and why.

Deliverables

- ☐ Updated ingestion code.
- ☐ Watermark store schema and example contents after two runs.
- ☐ Short explanation of the watermark and late-arriving data strategy.

Task 4: Product and Platform Judgment Note

Estimated time: 20-30 minutes

Write a short design note, approximately one page. This is used to understand how you think beyond the immediate coding task.

Requirements

- ☐ What assumptions would you challenge before productionizing this pipeline?

- ☐ What would you monitor in production, and what alerts would you create?
- ☐ How would you make this reusable for the next 10 APIs?
- ☐ What would make daily_account_summary a trustworthy data product for analysts and product managers?
- ☐ How would you expose lineage, ownership, documentation, and data quality status to consumers?
- ☐ What trade-offs did you make because of the time limit?
- ☐ If you used AI tools, how did you use them and how did you verify correctness?

Deliverables

- ☐ A concise product/platform design note in README.md or docs/product_platform_note.md.

6. Suggested Repository Structure

```
senior-de-assignment/
  README.md
  ingestion/
    ingest_transactions.py
    incremental_ingest.py
  dbt_project/                # if using dbt
    dbt_project.yml
    models/
      marts/daily_account_summary.sql
      marts/schema.yml
  sql/                       # if not using dbt
    daily_account_summary.sql
  tests/
  outputs/
    quarantine_sample.csv
    daily_account_summary_sample.csv
    watermark_run1.json
    watermark_run2.json
  docs/
    product_platform_note.md
```

7. README Checklist

- ☐ Setup and run instructions from a clean environment.
- ☐ Technology choice and rationale.
- ☐ Validation and duplicate-handling strategy.
- ☐ Incremental/watermark and late-arriving data strategy.
- ☐ Testing approach.
- ☐ Known limitations and what you would improve with more time.
- ☐ AI tool usage disclosure, if applicable.

8. Evaluation Criteria

Area	Weight	What strong submissions show
Pipeline correctness	25%	End-to-end execution, working pagination, reasonable retries, correct raw/quarantine/summary outputs.
Data quality and observability	20%	Invalid records captured with clear reasons, duplicates handled correctly, meaningful tests/assertions, monitoring thinking.
Incremental/idempotent design	15%	Watermark persists correctly, reruns do not duplicate data, late-arriving data strategy is explicit and defensible.
Code quality and reproducibility	15%	Readable, modular, minimal over-engineering, clear setup, no hardcoded secrets, sensible logging and errors.
Product/platform judgment	15%	Clear thinking on assumptions, users, ownership, lineage, reuse, operationalization, and trade-offs.
Agentic engineering discipline	10%	Effective AI tool usage if chosen, with independent verification and accountability.

9. Submission Notes

- ☐ A well-documented partial solution is better than a rushed complete solution that is difficult to run or reason about.
- ☐ Do not include credentials or secrets in your repository.
- ☐ Please include enough sample output for the reviewer to understand your results without rerunning everything immediately.
- ☐ Be ready to discuss your design choices, trade-offs, and any AI-assisted workflow you used during a follow-up interview.

Good luck. We look forward to reviewing your submission.